Khwezi Mining Monitoring Application

Pseudocode and Charts

MINN2020A: Computer Programming for Mining

19 October 2026

1 Pseudocode

1.1 Engineering rules

Pseudocode 1 Risk classification (*risk_classification*)

Input: likelihood L, consequence C (integers 1–5)
Output: score, level

```
score ← L × C
if score ≤ 4 then level ← "Low"
else if score ≤ 9 then level ← "Medium"
else if score ≤ 16 then level ← "High"
else level ← "Critical"
return score, level
```

Pseudocode 2 Equipment condition (*equipment_condition*)

Input: temperature T (°C), vibration V (mm/s)
Output: condition

```
if T > 100 or V > 8 then return "Critical"
else if T ≥ 80 or V ≥ 5 then return "Warning"
else return "Normal"
```

Pseudocode 3 Availability (*availability_percent*)

Input: operating hours O, downtime D
Output: availability (%)

```
total ← O + D
if total ≤ 0 then return 0
else return O/total × 100
```

1.2 Access control

Pseudocode 4 First run and login (*create_first_admin*, *login*)

```
seed demonstration CSV files if they do not exist
users ← load users file
if users is empty then                                              ▷ first run
    read username, password, confirmation
    if username empty or length(password) < 8 or passwords differ then
        show error; return
    salt ← 16 random bytes (hex)
    hash ← PBKDF2-HMAC-SHA256(password, salt, 300 000 iterations)
    save user with role "administrator"
read username and password
account ← user matching username (case-insensitive)
if account not found then show "Username or password is incorrect"; return
entered ← PBKDF2(password, account.salt)
if entered ≠ account.hash (constant-time compare) then show error; return
if account.role not in permission table then show error; return
set current user; build main window
```

Pseudocode 5 Permission check and role-filtered interface (*has_permission*, *build_app*)

```
function HasPermission(permission)
    return user is signed in and permission ∈ PERMISSIONS[user.role]

for all (tab title, required permission, builder) in tab list do
    if HasPermission(required permission) then
        create tab page; call builder to fill it
data getters (workers, incidents, equipment, maintenance) return an empty table when permission is missing
```

Pseudocode 6 Add user (*add_user*)

```
if not HasPermission("manage users") then show "Access denied"; return
read username, temporary password, role
if username empty or length(password) < 8 then show error; return
if username already exists (case-insensitive) then show error; return
generate salt and hash; append user; save users file; show confirmation
```

1.3 Alerts and dashboard

Pseudocode 7 Alert generation (*collect_alerts*)

alerts ← empty list
if HasPermission("worker safety data") **then**
    for all worker in workers **do**
        if PPE < 80 **then** add WARNING "PPE compliance below 80%"
        if fatigue ≥ 8 **then** add WARNING "high fatigue"
        if risk score ≥ 17 **then** add CRITICAL "critical safety risk"
        else if risk score ≥ 10 **then** add WARNING "high safety risk"
if HasPermission("safety incidents") **then**
    for all incident in incidents **do**
        if severity is "critical" **then** add CRITICAL "critical incident"
        if type is "observation" **and** severity ∈ {high, critical} **then** add WARNING "high-risk observation"
if HasPermission("equipment data") **then**
    for all unit in equipment **do**
        condition ← *equipment_condition*(temperature, vibration)
        if condition ≠ "Normal" **then** add alert with condition, temperature, vibration
        if downtime > 20 **then** add WARNING "excessive downtime"
        if maintenance status is "overdue" **then** add CRITICAL "maintenance overdue"
if HasPermission("maintenance data") **then**
    for all work order not completed **do**
        if due date < today **then** add CRITICAL "work order overdue"
        else if due date = today **then** add WARNING "work order due today"
return alerts

Pseudocode 8 Dashboard (*build_dashboard*)

load workers, incidents, equipment (empty if not permitted)
metrics ← empty list
if worker permission **then** add total workers, mean PPE, count fatigue ≥ 8, high-risk observations
if incident permission **then** add safety incidents, near misses, high/critical incidents, open critical incidents
if equipment permission **then**
    add total equipment, available, under maintenance, mean availability
    add count of units whose condition is "Critical"
draw one KPI card per metric
line chart: group incidents by month; plot last 6 months
pie chart: count Normal, Warning, Critical units; draw one slice per non-zero count
alerts ← *collect_alerts*(); display alert list

1.4 Worker safety and incidents

Pseudocode 9 Add worker assessment (*add_worker_assessment*)

if not HasPermission("worker safety data") **then** show error; **return**
read all form fields
if Worker ID or name empty **then** show error; **return**
if Worker ID already exists **then** show error; **return**
if PPE $\notin [0, 100]$ **or** fatigue $\notin [1, 10]$ **or** any count < 0 **or** $L, C \notin [1, 5]$ **or** a value is not a number **then**
    show error; **return**
(score, level) $\leftarrow$ *risk_classification*(L, C)
append record with today's date; save workers file; refresh table; confirm

Pseudocode 10 Search, filter, add and analyse incidents

function RefreshIncidents
    df $\leftarrow$ all incidents
    if search text not empty **then** keep rows where any column contains the text (case-insensitive)
    if filter field $\neq$ "All" **and** filter value not empty **then** keep rows where field equals value
    show rows; display "Showing n of N incidents"

function AddIncident
    if no permission **then** show error; **return**
    read fields
    if location or department empty **then** show error; **return**
    if date not YYYY-MM-DD **or** time not HH:MM **then** show error; **return**
    $n \leftarrow 1$; **while** "INC-n" exists **do** $n \leftarrow n + 1$
    append record with ID "INC-n"; save; refresh; confirm

function IncidentAnalysis
    count incidents, then group and count by department, shift, type and severity
    display summary

1.5 Equipment and maintenance

Pseudocode 11 Add equipment (*add_equipment*)

if no permission **then** show error; **return**
read fields
if Equipment ID empty **or** already exists **then** show error; **return**
if any numeric field invalid or negative **then** show error; **return**
availability $\leftarrow$ *availability_percent*(operating hours, downtime)
append record; save equipment file; refresh table; confirm

Pseudocode 12 Maintenance work orders (*add_maintenance_order*, *complete_maintenance_order*)

```
function CreateOrder
    if no permission or no equipment exists then show message; return
    read equipment ID, dates, type, priority, assignee, notes
    if assignee empty or dates invalid or due date < reported date then show error; return
    n ← 1; while "MO-n" exists do n ← n + 1
    save order with status "Open"
    set equipment maintenance status to "Due"; save; refresh

function CompleteOrder
    selected ← chosen table row
    if none selected then show message; return
    mark order "Completed" and set completed date to today; save
    if equipment has no other open order then set equipment status to "Available"
    save equipment; refresh; confirm

function EquipmentNeedingReview
    select units with status in {due, overdue, under maintenance} or downtime > 20 or availability < 80
    attach condition from equipment_condition; display
```

1.6 Risk assessment, reports and external data

Pseudocode 13 Risk calculator (*build_risk*)

```
read L, C from selection lists
if L or C not a whole number in 1–5 then show error; return
(score, level) ← risk_classification(L, C)
display "Risk score: score/25" and level
if level is High or Critical then display "Immediate review is recommended" in red
```

Pseudocode 14 Reports and CSV export (*report_rows*, *export_dataset*)

```
rows ← empty list
for all data area (worker, incident, equipment, maintenance) the role may access do
    compute totals, averages, counts and grouped counts (by department, shift, type, severity, month, status)
    append (category, metric, value) rows
display rows as summary text

function ExportDataset(file, columns, label)
    if role lacks permission for label then show error; return
    if dataset empty then show message; return
    ask user for save path; write CSV; confirm
```

Pseudocode 15 External data analysis (*upload_external_data*)

category ← selected data type
if role lacks permission for category **then** show error; **return**
path ← file chosen by user; **if** none **then return**
read file by extension (CSV, Excel or JSON)
if file has no rows **then** show error; **return**
map column names to the application schema using an alias table; add missing columns as blank
if any required field (e.g. temperature and vibration for equipment) is blank in every row **then** show error; **return**
standardise dates; for workers, fill missing risk score and level from $L \times C$
compute category metrics using the same thresholds as the main program
display metrics and a preview of up to 250 rows (saved records are not changed)